

An evolutionary approach for generating software models: The case of Kromaia in Game Software Engineering

Daniel Blasco*, Jaime Font, Mar Zamorano, Carlos Cetina

Universidad San Jorge. SVIT Research Group, Autovía A-23 Zaragoza-Huesca Km.299, 50830, Zaragoza, Spain

ARTICLE INFO

Article history:

Received 28 April 2020

Received in revised form 20 July 2020

Accepted 24 August 2020

Available online 27 August 2020

Keywords:

Model-Driven Engineering
Search-based software engineering
Game Software Engineering

ABSTRACT

In the context of Model-Driven Engineering applied to video games, software models are high-level abstractions that represent source code implementations of varied content such as the stages of the game, vehicles, or enemy entities (e.g., final bosses).

In this work, we present our Evolutionary Model Generation (EMoGen) approach to generate software models that are comparable in quality to the models created by human developers. Our approach is based on an evolution (mutation and crossover) and assessment cycle to generate the software models. We evaluated the software models generated by EMoGen in the Kromaia video game, which is a commercial video game released on Steam and PlayStation 4. Each model generated by EMoGen has more than 1000 model elements.

The results, which compare the software models generated by our approach and those generated by the developers, show that our approach achieves results that are comparable to the ones created manually by the developers in the retail and digital versions of the video game case study. However, our approach only takes five hours of unattended time in comparison to ten months of work by the developers. We perform a statistical analysis, and we make an implementation of EMoGen readily available.

© 2020 Elsevier Inc. All rights reserved.

1. Introduction

Game Software Engineering (GSE) is a research area that was compared with classic Software Engineering for the first time by McShaffry in 2003 (McShaffry, 2003). A recent survey, published in 2010, showed an overview of GSE research works and described the increasing interest in GSE (Ampatzoglou and Stamelos, 2010). Until now, GSE works have focused on issues like Requirement Traceability, but, due to the newness of this area, there are fields of study that remain unexplored.

As the survey mentioned above showed, Model-Driven Engineering (MDE) applied to video games is uncommon and, in general, is focused on generating source code from pre-existing models (Reyno and Carsí Cubel, 2009). In the following years, subsequent works have continued focusing on the generation of source code from models (Núñez-Valdéz et al., 2017, 2013; Solís-Martínez et al., 2015; Usman et al., 2017).

Some of the above MDE works (Reyno and Carsí Cubel, 2009; Usman et al., 2017) use UML as the modelling language, one MDE work (Solís-Martínez et al., 2015) uses process models,

while the rest of them (Núñez-Valdéz et al., 2017, 2013) use Domain-Specific Modelling Languages (DSL). The major advantage of modelling languages is that models use concepts that are much less bound to the underlying implementation technology, like video game engines such as Unreal (Games, 1998) or Unity (U. Technologies, 2005), and are much closer to the problem domain (the content of the video game) related to most popular programming languages (e.g., C++) (Selic, 2003). This notion of “model” should not be confused with “mesh” or “polygon mesh”, which are terms used in computer graphics and video games for the visual representation of 3D shapes/geometry.

In this work, we present our Evolutionary Model Generation (EMoGen) approach to generate software models that are comparable in quality to the models created by human developers. Automatically generating human-competitive software models is a challenging task. Fully achieving it spans the creation of model elements, the initialization of their properties, and their relationships with each other. Moreover, the resulting models must be valid, which includes satisfying modelling constraints. Finally, the human-competitive aspect is only achieved if the resulting models are comparable to those produced by software engineers for the same task at hand.

To generate the software models, our EMoGen approach takes an initial population of software models as input. These initial models may be randomly generated or may also be models that

* Correspondence to: SVIT Research Group, Universidad San Jorge, Autovía A-23 Zaragoza-Huesca Km. 299, 50.830, Villanueva de Gállego, Zaragoza, Spain.
E-mail addresses: dblasco@usj.es (D. Blasco), jfont@usj.es (J. Font), mzamorano@usj.es (M. Zamorano), ccetina@usj.es (C. Cetina).

were previously generated by software engineers. Then, the genetic operations of EMOGen, which are mutation and crossover, evolve the population. Invalid models are fixed by means of repair operations. The evolved models are assessed by means of a fitness function. This evolution and assessment cycle is repeated until a stop condition is met. The output of EMOGen is a ranking of generated models.

The case study for our work are the game characters at the end of each stage of the video game Kromaia: final bosses. This video game was released worldwide in both physical and digital versions for PC and PlayStation 4. In the context of video games, bosses are particularly powerful adversaries that are generally much stronger than the rest of the enemies in the video game. Usually, the player must overcome them at the end of a stage or level. Three-dimensional space simulation titles, such as the case study, include content like: a spaceship controlled by a human player; architecture, buildings or celestial bodies; a repertory of bosses and basic enemies; and projectiles that are fired by both the human player and the enemies.

Fig. 1 shows this game content in the context of a Kromaia playing session. Each of the stages or levels involves flying from a starting point to a certain destination, and the player spaceship must reach the goal before being destroyed. The stage implies exploring floating structures, avoiding asteroids and finding items along the route, which is protected by basic enemies (Fig. 1, G): ships and creatures that try to damage the player unit by firing projectiles that damage the player spaceship (Fig. 1, E). Basic enemies vary in anatomy and weaponry, but are significantly weaker than the player spaceship, in terms of endurance and firepower. In case that the player manages to reach the destination, the final boss (Fig. 1, A) corresponding that stage appears, and it must be defeated in order to complete the stage. Bosses differ from basic enemies in some aspects:

- They are huge in comparison to the player spaceship or basic enemies. An average boss is 50 times larger than the player spaceship.
- Bosses tend to be more complex in terms of anatomical structure. They include mobile parts, and the nexuses that connect two parts may behave as joints or even strings.
- They use several powerful weapons that are distributed along the parts that form their structure or body. In contrast, basic enemies use one or two frontal weapons.
- Bosses include in their structure special parts, which are the only damageable elements of their bodies. The player must locate these parts and destroy them, since it is the only way to defeat a boss.

The final bosses of the video game Kromaia are specified with the Shooter Definition Model Language (SDML). SDML is a DSL model for the video game domain. Specifically, SDML defines aspects included in video game entities:

- The anatomical structure, including which parts are used in it, their physical properties, and how they are connected to each other.
- The amount and distribution of vulnerable parts, weapons, and defences in the structure/body of the character.
- The movement behaviours associated to the whole body or its parts.

This modelling language has concepts such as hulls, links, weak points, weapons, and AI components. The top of Fig. 2 depicts an excerpt of the SDML that specifies one of the bosses of Kromaia (see the bottom of Fig. 2). Each model element (e.g., Hull Head) instantiates a concept (e.g., Hull) of the modelling language in order to specify the boss. More can be learned about the

SDML model of Fig. 2 in the following video: <https://youtu.be/Vp3Zt4qXkoY>

Our evaluation considers different starting points for our approach (the initial population of software models), ranging from randomly initialized models to models generated by software engineers. Models generated by software engineers are the most promising starting points; however, they come at a cost for software engineers. In contrast, random models reverse the pros and cons. The baseline is Random Search, which in the past (Bergstra and Bengio, 2012) has proven to outperform more sophisticated algorithms, and is a common sanity check practice in the Search-based Software Engineering (Harman and Jones, 2001) community. To evaluate the results, we use measurements studied in the scientific literature of video games: Completion, Duration, Uncertainty, Killer Moves, Permanence, and Lead Change. We also perform a statistical analysis to provide evidence of the significance of the results.

The results show that our approach produces human-competitive software models for the content (final bosses) of a commercial video game. On average, these software models have about 1300 model elements. They can be produced in five hours of unattended time, which is a significant reduction in time compared to ten months of work by the video game developers, as the Kromaia's Version Control System shows.¹ What is specially relevant is that these human-competitive models are achieved in the most positive scenario for software engineers: the seeds are random models. This means that software engineers do not have to manually generate the initial population of software models.

This is a step forward for Genetic Programming (Koza, 1992), where programs are evolved to fit a specific task, in the context of MDE. This paper contributes to the rise of what we call Genetic Modelling, and in this case, models are evolved for video game content in the particular case of our evaluation. Furthermore, this acceleration of video game content generation is also relevant for software developers of video games since they face the challenge of what is called the age of crunch (IGDA, 2018). There is an ever-increasing high demand for game content that is derived from early access releases, post-launch updates, downloadable content, and games as a service.

We make an open-source implementation² of EMOGen available as well as two model examples to facilitate the reproduction of the results. Even though this implementation is adapted to Kromaia, our approach includes general ideas that could work in other domains, and therefore make EMOGen useful for encouraging Genetic Modelling.

The structure used in this paper is the following: Section 2 summarizes related works. Section 3 presents Model-Driven Engineering for Video Games. Section 4 describes our EMOGen approach. Section 5 deals with evaluation. Section 6 presents the discussion. Section 7 describes the threats to validity, and Section 8 presents the conclusion of the paper.

2. Related work

This work is about generating software models using our EMOGen approach. Our evaluation is in the context of the video game content (bosses) of Kromaia. Therefore, our EMOGen approach generates models of Kromaia bosses. In this section, we discuss: (1) works that address game software engineering from the MDE community; and (2) works that address video game content generation. Video game content generation is also known as procedural content generation in the literature. Finally, we present an analysis of the research gap.

¹ Confirmed by the developers: two hours per day, including the time spent by real players on tests.

² <https://bitbucket.org/svitusj/EMoGen>.

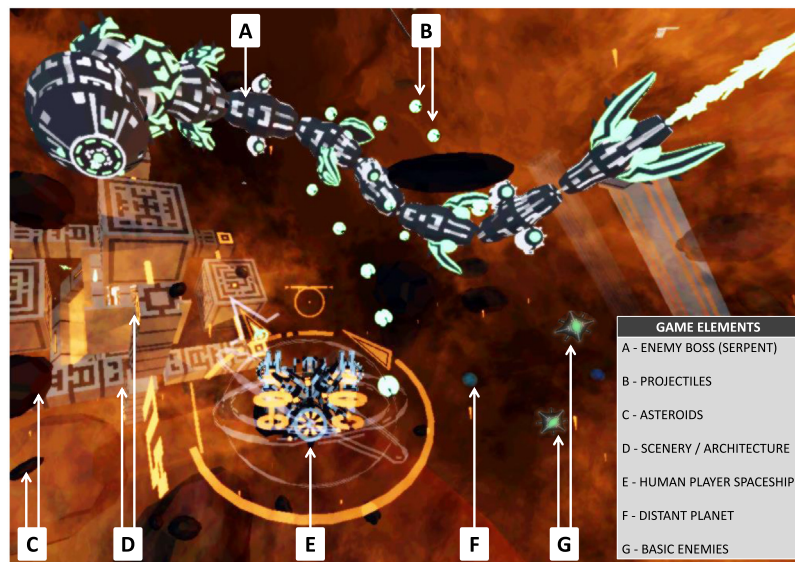


Fig. 1. Screenshot showing game content in Kromaia.

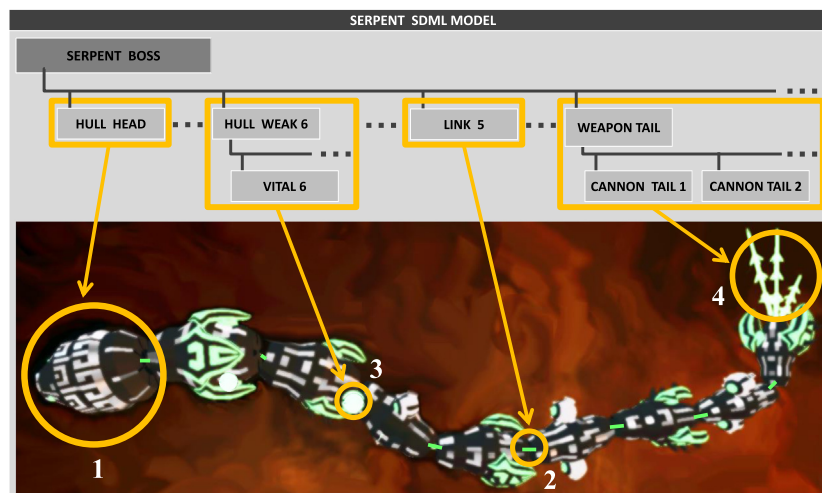


Fig. 2. SDML model of a boss (top) and the boss at run-time (bottom).

2.1. MDE and game software engineering

Platform independence is one of the potential benefits of using models as the main artefact for software development. The diversity of platforms that video game developers must deal with has motivated most of the research works that combine software models and the domain of video games.

The 2010 survey of Software Engineering Research for Computer Games (Ampatzoglou and Stamelos, 2010) identified only one work that applied Model-Driven Development to video games (Reyno and Carsí Cubel, 2009). That work coined the term “Model-Driven Game Development” and presented a first approach to 2D platform game³ prototyping through Model-Driven Development. Specifically, they used UML classes and state diagrams that were extended with stereotypes, and a model-to-code transformation to generate C++ code.

The research by Núñez-Valdéz et al. (2017, 2013) presents model-driven approaches that are intended to minimize errors,

time, and cost in multi-platform video game development. The work in Núñez-Valdéz et al. (2017) proposes a Domains-Specific Language, named Gade4all, and focuses on tablet and smartphone-oriented games. Solís-Martínez et al. (2015) suggest the use of business process models as the modelling language for video games. Specifically, they focus on the logic behind game loops in mobile games. In another work, Usman et al. (2017) propose a model-driven product-line approach that focuses on multi-platform (Android and Windows Phone) mobile game development and maintenance. They use a feature model to configure the UML use-case, class, and state machine diagrams.

Although the details are different, the above works share a common assertion: in the domain of video games, automated code generation from software models has the potential to significantly reduce the development effort and cost. Paradoxically, platform independence is an issue that is being addressed by widely used technologies such as Unity (U. Technologies, 2005) and Unreal Engine (Games, 1998) and is leading developers to be less concerned with this issue in this particular domain.

In the intersection between software models and evolutionary computation, Williams et al. (2011) use an evolutionary algorithm to search for desirable game character behaviours in a text-based

³ One of the first genres in video game history. In platform games, the main character climbs and jumps between suspended platforms while avoiding enemies/obstacles.

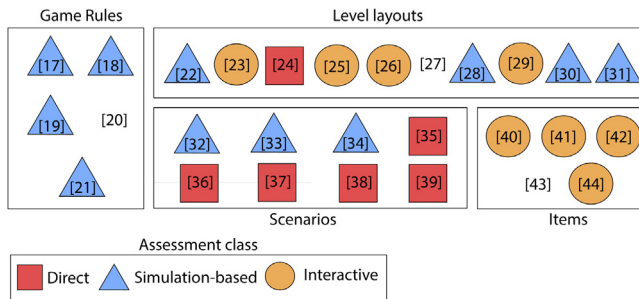


Fig. 3. Overview of the PCG related work.

video game that plays unattended combats and that outputs an outcome result. The character behaviour is defined using a Domain-Specific Language. The combats are managed internally and are only driven by behaviour parameters, without taking into account a spatial environment, real-time representation, or visual feedback (which takes into consideration the physical interaction of the characters, variation in the properties, etc.). However, the case study is a simplified text game. In addition, Williams et al. (2011) deals with game parameter adjustment, that is, the work does not address the generation of software models.

Another work that focuses on the intersection between software models and evolutionary computation is Avida-MDE (Goldsby and Cheng, 2008), which generates state machines that describe the behaviour of one of the classes of a software system (Adaptive Flood Warning System case study). The resulting state machines comply with developer requirements (scenarios for adaptation). Instead of generating whole models, Avida-MDE extends already existing models (object models and state machines) with new state machines that support new scenarios. The work in Goldsby and Cheng (2008) does not report the size of the generated state machines; however, the ones shown in the paper are around 50 model elements, which is significantly smaller than the more than 1000 model elements of the models of a commercial video game such as Kromaia.

2.2. Procedural content generation

Fig. 3 shows the works of the video game research community that address procedural content generation (PCG). All of these works generate part of the content of video games using either evolutionary computation (15 of 28) or machine learning (8 of 28). They generate content for the following parts of games.

Game rules (Hom and Marks, 2007; Togelius and Schmidhuber, 2008; Browne, 2008; Smith and Mateas, 2010; Salge and Mahlmann, 2010). These are the core of the game and changing them could result in a new game. To generate game rules, research works combine rules from existing games, such as Checkers or Pac-Man. The results of these works are mainly obtained at the scale of board or grid-based games.

Level Layouts. These are generated by combining different pre-existing design elements of levels, such as terrain, platforms, items, non-player characters. Research works achieve results at the scale of games such as a clone of Super Mario Bros, which is adapted for research, or Quake, a shooter game. Togelius et al. (2006), Pedersen et al. (2009), Sorenson and Pasquier (2010), Jennings-Teats et al. (2010), Sorenson et al. (2011), Van Der Linden et al. (2013), Roberts and Chen (2014), Raffae et al. (2014), Nogueira-Collazo et al. (2015) and Lelis et al. (2017)

Scenarios. These cover the structure of both puzzles (Oran-chak, 2010; Ashlock, 2010; Smith and Mateas, 2011) and maps/terrains (Ashlock et al., 2005; Frade et al., 2010; Togelius et al.,

2010; Loiacono et al., 2011; Ashlock et al., 2011). These research works have been applied in grid-based puzzles. In the case of maps/terrains, these works have been applied in grid-based maps and heightmap terrains.

Items (Hastings et al., 2009; Martin et al., 2010; Hastings and Stanley, 2010; Tutenel et al., 2011; Risi et al., 2015) include content such as weapons or buildings. Items are mostly generated by varying the properties of the items themselves. Research works create similar, but different, items in order to enrich players' experience. These works achieve results at the scale of games such as Galactic Arms Race, which is an indie development.

Furthermore, in Fig. 3, we classify the works in relation to their type of assessment following the Togelius' classification. Togelius et al. (2011) classified assessment as direct, simulation-based, or interactive. Direct assessment is depicted as a red square in Fig. 3 and uses features from the generated content to obtain a fitness value. Simulation-based assessment is depicted as a blue triangle and is based on artificial agents playing part of the game to evaluate the content. Finally, interactive assessment, which is depicted as an orange circle, involves the participation of real players scoring their gameplay explicitly or implicitly. The works with no defined assessment criteria could not be classified because they are surveys (Van Der Linden et al., 2013; Summerville et al., 2018) or they are not explained (Smith and Mateas, 2010; Tutenel et al., 2011). They are represented in Fig. 3 without a geometric classification.

2.3. Analysis of the research gap

There is a trend at the intersection of MDE and game software engineering (see Section 2.1) where research works focus on achieving platform independence by means of the abstraction of software models. These works propose automation to generate the implementation code for different platforms from the models. However, none of these works have explored the generation of software models in the context of video games. In that context, generating software models results in generating game content.

In the video game research community (see Section 2.2), research works explicitly address the generation of game content. So far, these works have succeeded in varying properties (works on items) and recombining pre-existing assets (works on game rules, level layouts, and scenarios). However, none of the works have leveraged models to generate game content.

Our work explores the gap of generating game content by leveraging software models. Our EMOGen approach evolves models and guides the evolution with a fitness function that uses the model interpreter for validation purposes and a game simulation which includes domain knowledge regarding the rules, the mechanics, and the course of a playing session. Leveraging a model interpreter to generate content is one of the differences between our work and the previous works.

Our work achieves successful results at the scale of contemporary video games such as Kromaia, while none of the previous works address contemporary games, and most of them address academic mobile games. Compared to content generation works, our work addresses a different part of games: final bosses. Our work is not limited to varying properties as previous works on item generation. Furthermore, our work could be applied in settings where previous assets are not available: works on game rules, level layouts, and scenarios require the existence of previous assets.

3. Model-driven engineering for video games: The Kromaia case

This section gives background on the role of models in Kromaia. Model-Driven Engineering (MDE) (Kent, 2002) aims to facilitate the development of complex systems by using models as the cornerstone of the software development process. Models are built in accordance with a metamodel that embodies the particularities and rules of a specific domain, formalizing what is valid and what is not when building a model for that metamodel. Models are used to formalize a system and capture each of its particularities. Then, those models can be used to reason about the system, perform validations, or transform it into different metalanguages, source code, or even run-time objects.

In the case of Kromaia, models are built against the Shooter Definition Model Language (SDML), a Domain-Specific Language created by Kraken Empire, which is the company that developed Kromaia. SDML allows for the definition of every element that will be present in the game, including worlds, vehicles, creatures, missions, enemies, etc. SDML is built using Ecore, the reference implementation of the Essential Meta Object Facility (EMOF) (Meta Object Facility, 2013), which is the standard metalanguage proposed by the Object Management Group (OMG) to build metamodels.

Kromaia was developed with a custom video game engine, created by the company, that acts as a framework in the context of the video game architecture, as shown in Fig. 4. This framework allows the developers to add new content in two different ways:

- **Programming**, making use of the Application Programming Interface (API) provided by the framework.
- **Software Models**, which are created using SDML and translated to its programming equivalent at run-time by an interpreter that is used by the engine (shown in Fig. 4).

The bottom of Fig. 2 shows a final boss that is included in the video game case study. Examples of the SDML concepts used in bosses are the following:

Hulls and Links: Hulls (see circle 1 of Fig. 2) are rigid bodies or solid objects that shape the structure of entities such as bosses. Hulls are connected via configurable nexuses, called links (circle 2 of Fig. 2). Hulls and links define both the anatomical hierarchy and physics for the boss. Through different arrangement and flexibility settings, links determine whether a boss includes mobile structures, rigid parts, or even complex limbs that resemble tentacles.

Weak Points: Weak points (circle 3 of Fig. 2) are concepts that are characterized by the fact that they can be damaged. Weak points are attached to hulls and they could optionally be arranged in layers to be unlocked as the player, who is the opponent of the boss, destroys them.

Weapons: Weapons (circle 4 of Fig. 2) are objects that could inflict damage on direct contact, firing bullets, launching smart homing projectiles, or tracing rays/beams. These four kinds correspond to the weapon types used in Kromaia. These weapons automatically aim at targets (human players) since they involve AI behaviours.

AI components: The behaviour patterns shown by bosses during a battle are defined by Artificial Intelligence components. These elements do not have a graphical representation, so they are not highlighted in Fig. 2. An AI module included in a boss may involve one or more of these concepts, suiting different battle situations or describing flocking behaviours.

The creation of game content in Kromaia is performed across four different stages using the concepts of SDML: Creative Design, Spatial Organization, Behaviour Specification, and Equipment Balance.

Creative Design: This is out of the scope of this work. Creative Design considers decisions from an artistic point of view. Therefore, it also involves concept art, texturing, and colour palette selection, since the design must adjust to the art direction. This Creative Design is mostly related to texture files, which are a few of the properties of some elements of SDML.

What our work does consider for the case study are the following technical stages of bosses that are addressed by means of SDML.

1 Spatial Organization: The specification for the anatomy that characterizes a boss is defined at this level. Spatial Organization produces a hierarchy that makes bosses resemble chains, trees, rings, quadrupeds, bipeds, and an unlimited range of structures that are similar to those examples or that are combinations of them. This specification makes the hierarchy possible since it includes the hull set that is present in the boss and the links that connect them, which may vary in nature and use (e.g., ropes or fixed joints).

2 Behaviour Specification: At this point, it is assumed that the spatial organization for the boss is complete since Behaviour Specification revolves around the means that the boss will use for moving between target locations, exploring the environment, chasing enemies, or dodging attacks. Therefore, anatomical constraints may not be compatible with certain behaviours. During this stage, the developers assign different artificial intelligence behaviours in order to match game experience needs and agility requirements.

3 Equipment Balance: The last stage focuses on weak point and weapon distribution. Both the user experience and the difficulty associated to the boss are heavily influenced by the inclusion of different defence/attack items and the hulls to which they are attached. In addition, the weapon and weak point distribution affects and limits the possible or even valid strategies that human players could adopt to try to defeat the boss.

Even without Creative Design, the generation of boss models poses a challenge that exceeds the capabilities of systematic approaches. A boss model, without considering Creative Design, requires more than 1000 model elements. In an optimistic scenario where properties are ignored and model elements can be enabled or disabled, the resulting search space has 2^{1000} different possibilities. Trying to assess every single model is not feasible, and, therefore, our EMOGen approach relies on an evolutionary algorithm to explore the search space.

4. Our EMOGen approach

This section presents our EMOGen approach, which leverages evolutionary computation to generate human-competitive software models. Fig. 5 shows the Evolutionary Algorithm (EA) that evolves a population of software models (models that follow our encoding) through genetic operations. First, the initial seeds are used to generate an initial population. Then, the population is assessed using the fitness function. Next, the population is evolved by applying genetic operators. This process (assessment + evolution) is repeated until the stop condition is met. Then, the population is decoded into models that are ready to be used.

When applying EMOGen to the Kromaia case study, we encode the models for the final bosses that are faced at the end of each level that are present in the video game. These models include the Spatial Organization, the Behaviour Specification, and the Equipment Balance of each of the final bosses (see Section 3).

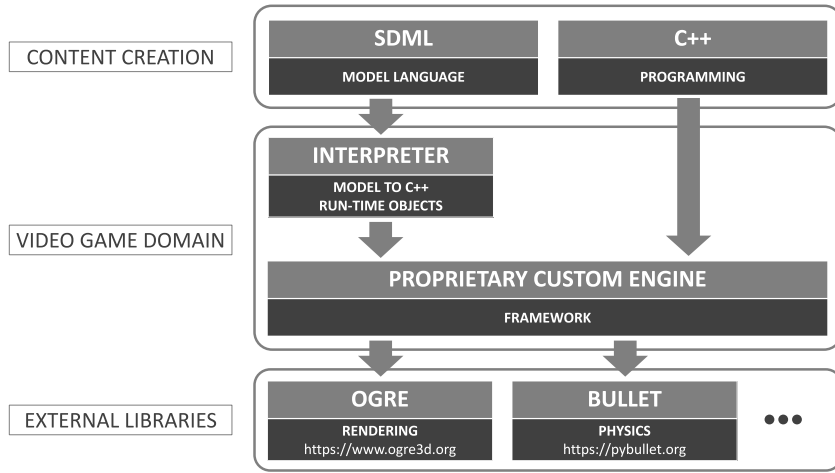


Fig. 4. The different architecture layers in Kromaia, the video game case study.

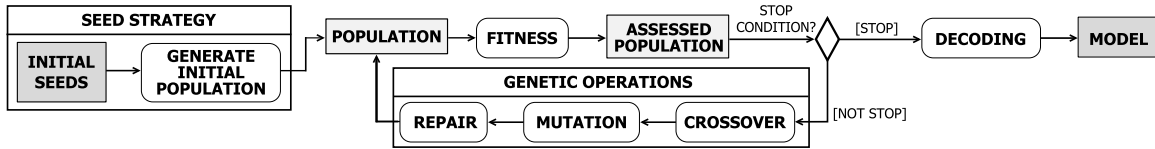


Fig. 5. Overview of the EMOGen approach.

4.1. Fitness of the EMOGen approach

The objective of the fitness function in our EMOGen approach is to assess the quality of each individual as a model. This is done by taking into account the validity of the model and a game simulation that includes Domain Knowledge:

Validity: First, our approach checks the model in search of inconsistencies that would lead to classifying it as not being valid for use, hence assigning a fitness value of 0.

Domain Knowledge: Once the models are considered valid, our approach determines the suitability of the model. This is done by using a game simulation that takes into account domain knowledge that is related to the elements present in a battle that involves a human player and a boss. It considers aspects such as the weaponry used by each of them and the differences between the two entities in terms of agility, speed, endurance, or size.

When applying EMOGen to the Kromaia case study, the validity of the models is performed by a run-time interpreter that is part of the game. The boss models generated by either human designers or our approach may not be valid due to inconsistent data that is related to the different stages (see Section 3). For instance, it is not valid to indicate in the model that a certain hull is connected to another hull that is not even present in that model. It is also invalid to denote a behaviour leading role for a hull that does not have at least one weapon attached. In these cases, the model would be assigned a fitness value of 0.

When no validation errors are found for a certain boss model and it is confirmed as valid, its fitness value is obtained from a simulation that reproduces a duel between the boss of the model and a human player. During that simulation, the player faces the boss in order to destroy the weak points that are available at that moment, whereas the boss acts according to the anatomy, behaviour, and attack/defence balance that is included in its model, trying to defeat the player. In that simulation, both the boss and the human player try to win the match and do not avoid confrontation, try to prevent draw/tie games, and try to ensure that there is a winner. The fitness value is calculated

once the simulation process is finished, and our approach collects information on the battle progress and key events.

The information retrieved from the simulation is the data that the developers regard as relevant, using their domain knowledge, for determining whether or not a boss is suitable for a commercial release of the video game, i.e., the percentage of human player victories ($F_{Victory}$) and the percentage of human player health left once the player wins a duel (F_{Health}). The *clamp* function is used in the fitness measures:

$$clamp_{[0,1]}(x) = \max(0, \min(x, 1)) \quad (1)$$

In our approach, $F_{Victory}$ is calculated as a measure of the difference between the number of human player victories (V_p) and the optimal number of victories (33%, according to the developers of Kromaia and their criteria) ($V_{Optimal}$):

$$F_{Victory} = clamp_{[0,1]} \left(1 - \frac{|V_{Optimal} - V_p|}{V_{Optimal}} \right) \quad (2)$$

The criterion F_{Health} , which refers to completed duels that end in human player victories, is the average difference between the human player's health percentage once the duel is over (Θ_p) and the optimal health level that the player should have at that point ($\Theta_{Optimal}$, 20%, according to the developers):

$$F_{Health} = clamp_{[0,1]} \left(1 - \frac{\sum_{d=1}^{V_p} \frac{|\Theta_{Optimal} - \Theta_p|}{\Theta_{Optimal}}}{V_p} \right) \quad (3)$$

$F_{Overall}$ is an average fitness value for a boss model that includes the fitness criteria described above and validation information, with *Validity* being a value that determines whether or not a model is valid (1 and 0, respectively):

$$F_{Overall} = \min(Validity, \frac{\sum_{i=1}^N F_i}{N}) \quad (4)$$

In the end, $F_{Overall}$ is a value in $[0, 1]$ that is used to assess a boss model when our EMOGen approach is applied to the Kromaia case study.

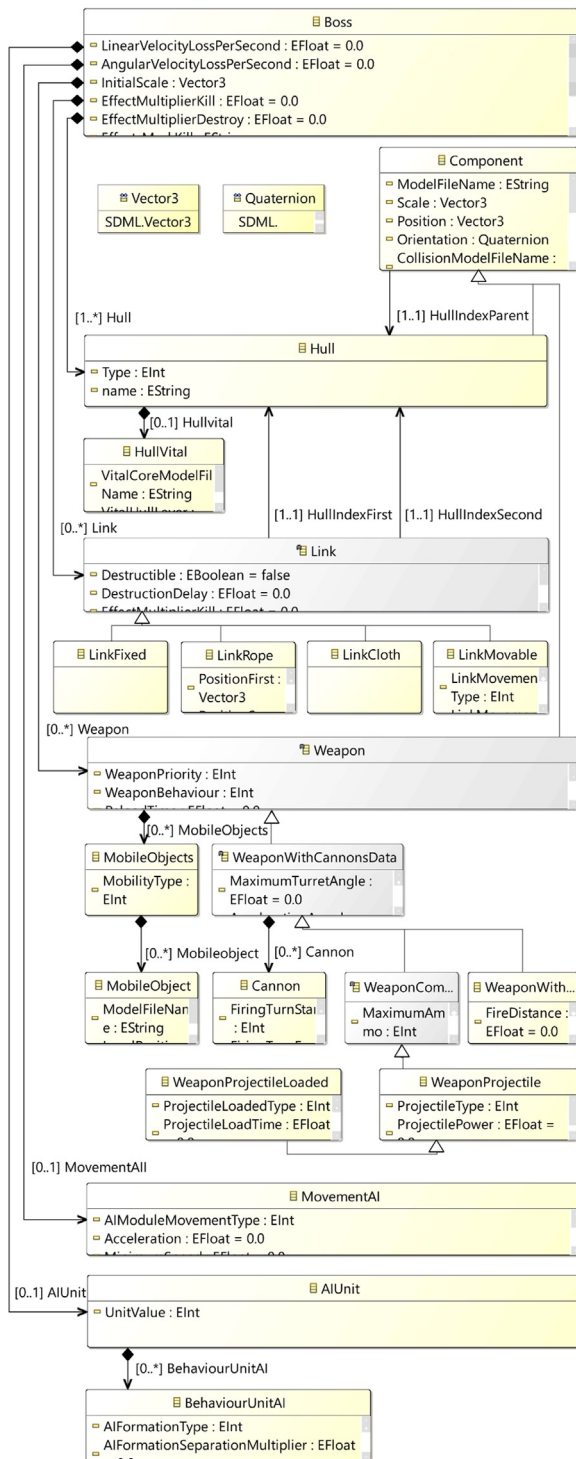


Fig. 6. Excerpt of the Metamodel of the boss models.

4.2. Model encoding of the EMOGen approach

In evolutionary algorithms like the one used by our EMOGen approach, the models are encoded, and this representation is usually achieved in evolutionary algorithms with arrays or strings. In this work, we encode models elements in a way similar to our previous works (Font et al., 2016; Arcega et al., 2018, 2019) for models of the Induction Hob and Train Control domains.

ENCODING		HULLS				
		H 0	H 1	H 2	H 3	...
ELEMENTS AND PROPERTIES	ENABLED HULL	0	1	1	1	...
	LINK PARENT HULL	0	-1	4	2	...
	BEHAVIOUR LEAD	0	1	0	0	...
	GUN TURRET	1	0	1	1	...
	WEAK POINT	0	0	1	1	...
	...	...	...	...	...	...

Fig. 7. Example of encoding for boss models.

When applying EMOGen to the Kromaia case study, the boss models contain elements, such as hulls and weapons, that are defined as being present or absent throughout the different stages in the creation process as well as properties that are constrained to a range of values. Fig. 6 shows an excerpt the metamodel which the boss models are produced in accordance with. This excerpt omits secondary concepts, relationships and properties that are not as common or relevant as those presented in the figure. The metamodel contains more than 20 concepts, over 20 relationships and more than 60 properties. A final boss model like the example in Fig. 2 and in the example video for this research (<https://youtu.be/Vp3Zt4qXkoY>) contains around 1300 elements.

Our approach encodes boss models as bi-dimensional matrices, in which columns correspond to the hulls that could be used in the model and in which each row indicates present or absent elements as well as properties:

Elements: Fig. 7 shows four of the elements represented in our encoding. The presence or absence of these elements is defined through binary values (1 and 0, respectively). For instance, in the example shown in Fig. 7, Hull10 would not be present in the model and Hull12 would have a turret.

Properties: The second row in Fig. 7 shows a row with non-binary values that correspond to link relationships. This property indicates the parents that the hull is linked to. For instance, H3 of Fig. 7 means that Hull12 and Hull13 are linked in a way that Hull12 acts as a parent in the hierarchy. In addition, the encoding used by our approach represents hulls that do not depend on other hulls via links (root hulls) with values of -1 for that property, as shown in Fig. 7.

4.3. Genetic operations of the EMOGen approach

Our EMOGen approach generates new models using some of the existing ones in the population as parents. This process is supported by genetic operators that are adapted to work with the EMOGen encoding that represents models.

First, it is necessary to select the parents from the model population before applying the genetic operators. The fittest of the potential parents are selected using the fitness value calculated for each model in the population.

Crossover: The crossover operation mixes the content of two models to create a new one. The new model takes a first random half with size n from the first parent and a second half with size $S - n$ from the second parent, with S being the total size of the model.

Mutation: This operator is named after the mutations found in biology. These mutations make individuals show non-inherited modifications in their genes due to random factors. In our EMOGen approach, the mutation operation is applied on the new models that are created through crossover operations; however, changes depend on a certain probability, so they do not always occur. Due to the nature of the encoding used by our approach, mutations add and remove elements from the model and change properties.

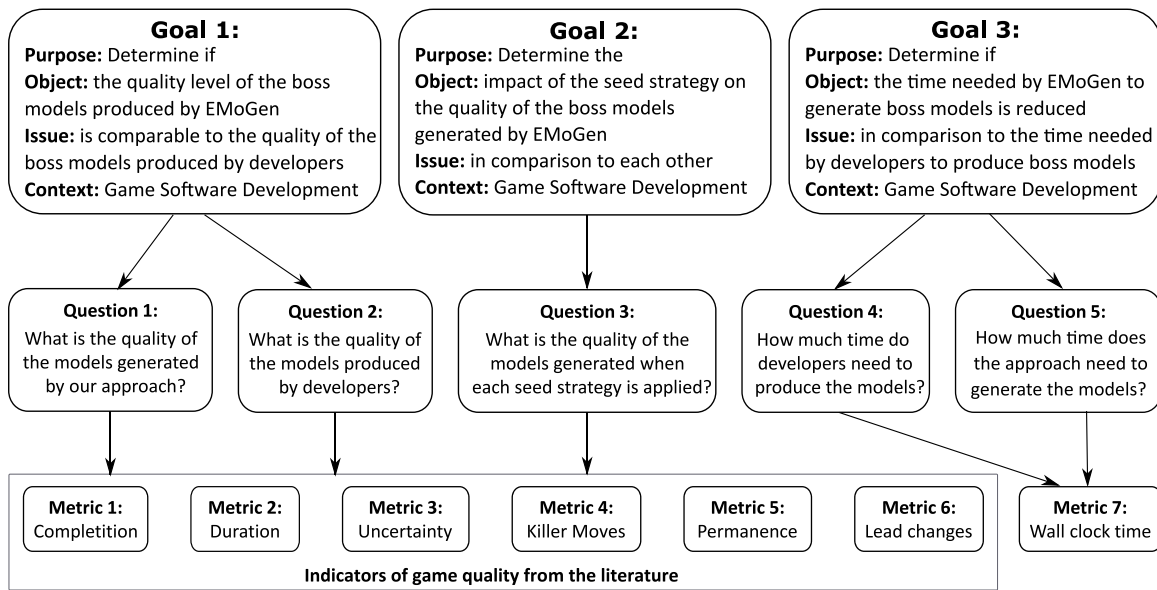


Fig. 8. Application of the goal-question-metric method for the evaluation.

Repair: Finally, after crossover and mutation have modified the genetic material of the individuals, inconsistencies may appear. For instance, when the crossover operation is applied, a link to a hull can be “broken”, resulting in a new individual that is pointing to a hull that is not activated. Inconsistencies of this kind will prevent the model from being loaded into the game since it will fail to pass the model interpreter validation. The repair operator mitigates inconsistencies, making small modifications to the individuals, like modifying links that point nowhere. We do not claim to have a complete catalogue of repairs that guarantees that the resulting model will be accepted by the model interpreter.

5. Evaluation

This section presents the evaluation performed to determine if EMOGen can help game developers when creating the models for video games. In past works, there are four types of studies explained by Basili (1996) and Travassos and de Oliveira Barros (2003). They refer to in-silico, in-vivo, in-vitro, and in-virtuo. More recent works used models as experimentation units (Shull et al., 2010) within in-virtuo experiments (Arcega et al., 2017), but in this work we perform an in-silico experiment, in order to minimize the interaction with humans, and, therefore, favour the replicability of the study (Travassos and de Oliveira Barros, 2003).

We defined the experimental design of the evaluation following the Goal-Question-Metric (GQM) (Basili, 1996) method. The GQM method defines a measurement model on three levels: a set of goals (the conceptual level) defined through a set of questions (the operational level) that can be answered through a set of metrics (the quantitative level). Following the template proposed by Basili et al. (1994), we set three goals, which are defined through five questions that can be answered using seven metrics (see Fig. 8). Apart from wall clock time, which is necessary to evaluate the time needed by our approach and the developers to generate models, a set of indicators of game quality described and widely used in the literature of video game research are used to answer the questions. We use the six indicators that Browne et al. studied and recommended for being the most relevant (Browne and Maire, 2010). Specifically, Browne et al. correlated 57 different quality indicators with players rankings. At the end, six of them stand out as the most important: Completion, Duration, Uncertainty, Killer Moves, Permanence, and Lead Change. Each of

those metrics is measured using the characteristics of the case study. The suitability of a boss model is assessed studying the data obtained from a duel between the boss and a simulated player. That data provides values that are used in order to measure the metrics: the duration of a duel, the player victory percentage, the amount of relevant events in a match, and the health level of the player after the end of a duel.

Goal 1 is to determine if the quality level of the boss models produced by EMOGen is comparable to the quality of the boss models produced by developers. To determine this, we define two questions: Q1—What is the quality of the models generated by EMOGen?, and Q2—What is the quality of the models produced by developers?. The quality will be assessed using a set of six metrics that are widely used in the literature to assess the quality of games.

Goal 2 is to determine the impact of the seed strategies on the quality of the boss models generated by EMOGen in comparison to the quality of the boss models produced by the developers. The seeds are boss models that the evolutionary algorithm in our approach is fed with as starting points. To determine this impact, we define a new question: Q3—What is the quality of the models generated when each seed strategy is applied?. The quality level will be assessed as with Q1 and Q2, using six metrics from the literature.

Goal 3 is to determine if the time needed by EMOGen to generate boss models is reduced in comparison to the time needed by the developers to produce boss models. To determine this, we define two questions: Q4—How much time do developers need to produce the models? and Q5—How much time does EMOGen need to generate the models?. These two questions will be assessed by measuring the wall clock time.

The following subsections present a description of the experimental setup, the metrics used to answer the questions, the details of the implementation, and the results.

5.1. Experimental setup

Fig. 9 shows an overview of the evaluation process followed. The first step of the evaluation is the extraction of information from the oracle that is provided by the developers of Kromaia (see top-left of Fig. 9). The developers provided the set of final boss models and a set of seeds that is used by the approach to

generate the initial population. Specifically, we use two types of seeds:

Final Boss: The type of boss that the player can find at the end of the level. Each final boss contains around 1300 model elements. The developers provided five different final bosses.

Miniboss: Enemies with less relevance in the game than a final boss but that are also built following the same stages and language (SDML). Each Miniboss contains around 500 model elements. The developers provided five different Minibosses.

Then, we perform a sanity check; we execute a random search to determine if the search space is large enough to benefit from the application of an evolutionary algorithm such as the one proposed here or a simple random search that is able to yield good results. To ensure a fair comparison, the random search is allocated with a budget that is similar to the one used by our approach. Specifically, the budget is in terms of the number of times the fitness function is executed as suggested in the literature (Johnson, 2002).

Our approach is executed seven times, using a different seed strategy each time:

100R: The whole initial population is randomly generated so the seeds from the oracle are not used in this execution.

1F: A single final boss is randomly selected, out of the 5 available, and provided as initial seed. To generate the initial population, the seed is encoded as an individual and the rest of the population is obtained through the application of the mutation operation to the individual.

1M: A single Miniboss is randomly selected, out of the 5 available, and provided as initial seed. To generate the initial population, the seed is encoded as an individual and the rest of the population is obtained through the application of the mutation operation to the individual.

5F: The five final bosses are provided as initial seed. Similarly, the five final bosses are encoded as five individuals and the rest of the population is obtained through mutations of those five individuals.

5M: Similarly, the five Minibosses are provided as initial seed, encoded, and mutated to obtain more individuals and to complete the population.

95R+5F: The five final bosses are provided as initial seed. The five final bosses are encoded as individuals, but the rest of the population is randomly generated.

95R+5M: Similarly, the five Minibosses are provided as initial seed and encoded as individuals. The rest of the population is randomly generated.

As suggested in the literature (Arcuri and Fraser, 2013), each execution of the approach is repeated 30 times to compensate for the stochastic nature of evolutionary algorithms. Then, the resulting boss models are measured using the six Quality measurements (Thompson, 2000; Althöfer, 2003; Iida et al., 2004; Browne, 2005; Kramer, 2000; Gardner, 2004; Ellis, 1914; Birkhoff, 1933; Stiny and Gips, 1978) (see the middle part of Fig. 9). Similarly, the Boss Models obtained from the oracle are also subject to the same quality measurements. Then, all of the results are compared and statistically analysed to determine the significance of the results.

In order to define the mathematical expressions that represent each of those quality measurements for the game studied, Kromaia, different tests and surveys were conducted with more than 30 users who belonged to the main target audience of the commercial case study.

The company responsible for the development of Kromaia provided data from their Version Control System with the help of the two engineers who produced and modified the boss models until the versions included in the final product were completed. These engineers have worked in the video game industry for

15 years and were informed of the purpose of the present work. Additionally, they signed a consent form before the data was used in this research. The company and these engineers collaborated in this work in order to research on possible improvements in the boss production process.

5.2. Quality measurements

Previous research works have formalized fundamental and measurable indicators of game quality, like Depth and Decisiveness (Thompson, 2000), Tension (Kramer, 2000), Interestingness (Althöfer, 2003), Uncertainty (Iida et al., 2004), or Interaction (Browne, 2005). In a more recent research done by Browne et al. the experimentation with game users showed that the following criteria stand out as being the most important: Completion, Duration, Uncertainty, Killer Moves, Permanence, and Lead Change (Browne and Maire, 2010). Our evaluation measures these criteria with values in the interval [0,1].

Completion (Viability): A game against a boss unit should end with more conclusions (victories for either the player or the boss) than draws/ties. The criterion $Q_{Completion}$ calculates a ratio of conclusions over total duel count:

$$Q_{Completion} = \frac{Conclusions}{Duels} \quad (5)$$

Duration (Viability): The duration of duels between players and boss units is expected to be around a certain optimal value. For the video game case study, through tests and questionnaires with players, the developers determined that concentration and engagement for an average boss reach their peak at approximately 10 min ($T_{Optimal}$), whereas the maximum accepted time was estimated to be 20 min ($2 * T_{Optimal}$). Significant deviations from that reference value are good design-flaw indicators: short games are probably too easy; and duels that go on a lot longer than expected tend to make players lose interest. The criterion $Q_{Duration}$ is a measure of the average difference between the duration of each duel (T_d) and the desired, optimal duration ($T_{Optimal}$):

$$Q_{Duration} = clamp_{[0,1]} \left(1 - \frac{\sum_{d=1}^{Duels} \frac{|T_{Optimal} - T_d|}{T_{Optimal}}}{Duels} \right) \quad (6)$$

Uncertainty (Quality): In order to keep players engaged with a duel, neither the player nor the boss unit should get extremely close to victory or defeat too early before the duel is settled, with (T_d) being its duration. Therefore, a duel is considered to be more uncertain the longer the time until the player's or the boss unit's health levels reach a dangerous/critical status (P_d and B_d , respectively). For each duel, $Q_{Uncertainty}$ measures the average deviation between the time at which it is detected that one of the contenders is on the verge of defeat and the time corresponding to the duration of the duel.

$$Q_{Uncertainty} = clamp_{[0,1]} \left(1 - \frac{\sum_{d=1}^{Duels} \frac{T_d - \min(P_d, B_d)}{T_d}}{Duels} \right) \quad (7)$$

Killer Moves: Q_{KMoves} measures the proportion of killer moves by any contender (K), taking into account the moves that are considered to be remarkable highlights (H) but that are less important than killer moves. In the video game case study, the developers considered that a highlight move happens when either the boss unit or the player experiences a decrease in health; killer moves are those that make the difference in health between the contenders reach 30%.

$$Q_{KMoves} = clamp_{[0,1]} \left(1 - \frac{\sum_{d=1}^{Duels} \frac{K_d}{H_d}}{Duels} \right) \quad (8)$$

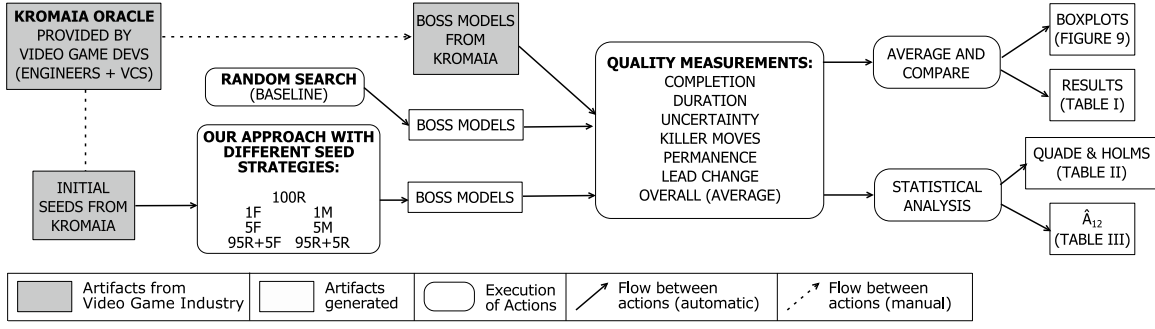


Fig. 9. EMOGen approach evaluation process.

Permanence: Duels with a high permanence value are games in which the advantages given by significant actions or moves by one of the contenders are unlikely to be immediately reverted by the opponent in terms of dominance. In the video game case study, the developers considered every highlight move and killer move to be meaningful actions, with recovery moves (R) being those that quickly cancelled the advantages given by other previous killer or highlight moves. The criterion $Q_{Permanence}$ is measured as follows:

$$Q_{Permanence} = clamp_{[0,1]} \left(1 - \frac{\sum_{d=1}^{Duels} \frac{R_d}{H_d + K_d}}{Duels} \right) \quad (9)$$

Lead Change: The lack of lead changes indicates low dramatic value. In the video game case study, the lead is determined at any given moment by considering the contender with the highest health level. This criterion is measured taking into account those highlight or killer moves that cause the lead to change (L) during the course of a duel:

$$Q_{LChange} = clamp_{[0,1]} \left(\frac{\sum_{d=1}^{Duels} \frac{L_d}{H_d + K_d}}{Duels} \right) \quad (10)$$

Our approach evaluated these six (N) criteria for each boss unit that is included in the commercial release of the video game case study in order to obtain a quality threshold that is useful for verifying whether the results obtained by our approach reach the same quality levels. $Q_{Overall}$ calculates an average quality value for a model, including all of the quality criterion studied:

$$Q_{Overall} = \frac{\sum_{i=1}^N Q_i}{N} \quad (11)$$

The above quality measure is used to determine how many of the models produced by our approach are comparable in quality to those present in the case study.

5.3. Implementation details

In order to implement the approach, we used the TinyXML parser to process SDML models. In addition, the specifications of the computer used in the evaluation process were the following: Toshiba Satellite Pro L830 laptop, with an Intel® Core™ i5-3317U processor with 4 GB RAM and Windows 8 64 bit.

For the parameters of the EA, since the focus of this work is not the tuning of parameters, we used values from the literature that have proven to provide good results with models (Font et al., 2016; Arcega et al., 2018, 2019). The mutation probability (p_m) depends on the number of hulls in the boss: $1/(\text{Hulls Number})$.

In general, there are two atomic types of performance measures that are used to evaluate search algorithms: measures regarding speed and measures regarding quality of the solution. Since the focus of this paper is on the quality of the solution, we

allocated a budget for each execution of the approach. Specifically, after running some prior tests, we identified the time of convergence, which is the point where the search reaches the peak and no further improvements occur, to be around 40 min of execution. To ensure convergence, the amount of wall clock time for each of the runs was set to one hour. A prototype of EMOGen can be found at: <https://bitbucket.org/svitusj/EMOGen>

5.4. Results

Fig. 10 shows the results of the execution of our approach for each of the seven combinations of seeds and population strategy (100R, 1F, 1M, 5F, 5M, 95R+5F, 95R+5M). The executions are grouped to show the performance for a specific quality measurement (Completion, Duration, Uncertainty, Killer Moves, Permanence, Lead Change). The last column, with shaded background, shows the average of all of the quality measures for each execution. In addition, each population strategy for a specific quality is crossed by a horizontal line that indicates the value obtained by the human-generated final boss models that were obtained from the Kromaia oracle (see top-left of Fig. 9).

Each boxplot is generated from the results of 30 executions (Arcuri and Fraser, 2013) where each execution yields 100 individuals as a result. Therefore, each boxplot represents 3000 values of a specific quality in a final boss model. Fig. 10 shows in each column how the quality values obtained for each of the seven strategies studied differ from the values for the models generated by the developers, which are represented by the horizontal lines that cross each column. The boxplots that are closer to the horizontal lines are more similar in quality to the models produced by the developers. Additionally, the use of boxplots allows for the representation of the different results for the strategies used.

Similarly, Table 1 shows the values obtained by each seed strategy (rows) and each of the quality measurements (columns). Each value is reported from 0 to 1, which are the worst and best possible values, respectively.

In addition, the first row shows the results for the sanity check: a Random search executed with a budget that is similar to our approach in terms of fitness executions (i.e., 3 million fitness executions). The purpose of the sanity check is to determine whether there is a need for a complex search strategy or the solutions to the problem can be found by mere chance. In our case, none of the individuals that were generated as part of the random search were able to be validated by our model interpreter; therefore, their score is 0.

The answer to Q1, which asks about the quality of the models generated by our approach, can be seen in the boxplots of Fig. 10 and in Table 1: they show the values of each of the metrics for the different seed strategies. Similarly, the answer to Q2, which asks about the quality of the models produced by developers, can be seen as the horizontal lines of Fig. 10, which cross each column

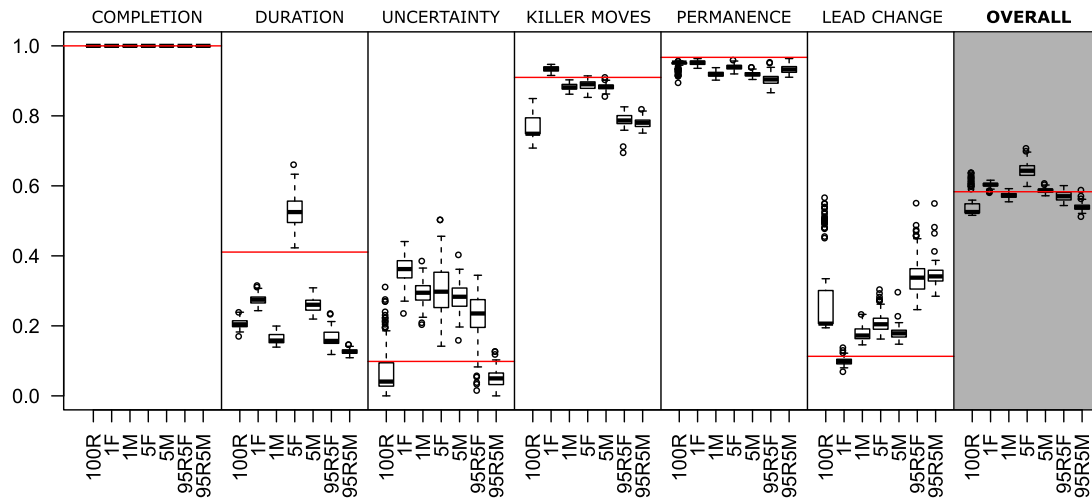


Fig. 10. The results of the application of EMOGen with seven different seed strategies to generate final boss models. The results are grouped based on the 6 quality measures; the horizontal line in each group represents the value obtained by the original final boss models from Kromaia.

10, and the associated values from Table 1 (last row, Kromaia Oracle). To address Goal 1, we compared the results obtained by our approach with those from the oracle. The values were similar, particularly in terms of the overall quality, with differences of around 5% at maximum. Therefore, we can conclude that the approach is able to generate final boss models that are comparable to those from Kromaia, the video game case study, whose final boss models were created manually by software engineers.

5.5. Statistical analysis

To answer Q3 and to compare the impact of each of the seed strategies on the quality of the results, the empirical data was analysed following the guidelines from the literature (Arcuri and Briand, 2014). The statistical analysis included a significance test, the corresponding post-hoc analysis, and an effect size measure.

5.5.1. Statistical significance

We applied a statistical test to the results of the seven seed strategies to determine if there were significant differences among the final boss models produced in terms of the quality measurements presented (i.e., the differences in the results were not obtained by mere chance).

After running the approaches a large enough number of times (30 as suggested by the literature Arcuri and Briand, 2014), we applied the Quade test since our data does not follow a normal distribution and the Quade test has proven (García et al., 2010a) to be better than the rest of the non-parametric tests when working with real data.

The Quade test results in a p -value between 0 and 1, with 0 indicating that there are significant differences among the different seeds strategies and 1 indicating that there are no such differences. The threshold accepted by the research community is 0.05 (Arcuri and Briand, 2014), meaning that p -values below that number are statistically significant.

The results of the Quade test give a p -value below the 0.05 threshold for all of the measurements ($\ll 2.2 \times 10^{-16}$), indicating that the differences observed in the results are significant enough to be caused by the seed strategy and are not due to mere chance. The test was not applied for the Completion quality measure since all of the results were 1 and there was no variance.

5.5.2. Post-hoc analysis

The Quade test only determined that there are differences among all of the seed strategies. To identify the specific seed strategies yielding significant differences, we applied a post-hoc analysis. This analysis consists of pair-wise comparisons of the results of each seed strategy to determine if there are statistically significant differences among the results of each pair of strategies.

We applied the Holm's post-hoc analysis, which is the most common post-hoc analysis applied after a Quade test (García et al., 2010b). Again to interpret the results, a value below 0.05 indicates that the differences between the two strategies are significant enough to be considered to be caused by the seed strategies. Table 2 shows the results of the post-hoc analysis. Each column shows the p -value for a specific quality measurement, while each row shows one of the pair-wise combinations of two seed strategies (the order does not matter for this test). Table 2 shows only the pairs of seed strategies that obtained values above 0.5, which cannot be considered significant enough to determine that the differences are due to the seed strategy. Pairs of strategies not shown in the table obtained values below 0.5 for all of the measurements, so the differences are due to the seed strategy applied.

For instance, the differences between 1M and 5M seed strategies cannot be considered significant enough for some of the measurements like uncertainty, killer moves, permanence, or lead change (see row labelled as 1M vs. 5M in the middle of Table 2). It is common to obtain these results because the differences between some pairs of seed strategies are subtle.

These results partially answer Q3. Taking into account the differences in the results from Table 1 and the fact that they are significant as shown by the Holm's post-hoc analysis (see Table 2), we can conclude that the selection of the seed strategy does have an impact on the quality of the final boss model produced in terms of the quality measurements included in this study.

5.5.3. Effect size

It has been proven that statistically significant differences can be obtained even if they are so small as to be of no practical value (Arcuri and Briand, 2014). To completely study Goal 2, we analysed the effect size to determine the magnitude of the improvement of one seed strategy over the others. To do this, we measured the Vargha and Delaney's $\hat{A}_{12}$ non-parametric effect size (Grissom and Kim, 2005; Vargha and Delaney, 2000). $\hat{A}_{12}$ can be used to measure the probability of one seed strategy

Table 1

Mean values and standard deviations for each of the quality metrics (columns), and each of the seed strategies (rows). The seed strategies that achieve the best and worst results for each metric are highlighted in grey.

	Completion	Duration	Uncertainty	Killer moves	Permanence	Lead change	Overall
Random search	0 ± 0	0 ± 0	0 ± 0	0 ± 0	0 ± 0	0 ± 0	0 ± 0
100R	1 ± 0	0.21 ± 0.01	0.07 ± 0.08	0.77 ± 0.04	0.95 ± 0.01	0.28 ± 0.13	0.55 ± 0.04
1F	1 ± 0	0.28 ± 0.02	0.36 ± 0.04	0.93 ± 0.01	0.95 ± 0.01	0.10 ± 0.01	0.60 ± 0.01
1M	1 ± 0	0.16 ± 0.02	0.30 ± 0.03	0.88 ± 0.01	0.92 ± 0.01	0.18 ± 0.02	0.57 ± 0.01
5F	1 ± 0	0.53 ± 0.05	0.30 ± 0.07	0.89 ± 0.01	0.94 ± 0.01	0.21 ± 0.03	0.64 ± 0.02
5M	1 ± 0	0.26 ± 0.02	0.28 ± 0.04	0.88 ± 0.01	0.92 ± 0.01	0.18 ± 0.02	0.59 ± 0.01
95R+5F	1 ± 0	0.17 ± 0.02	0.23 ± 0.07	0.79 ± 0.02	0.90 ± 0.02	0.34 ± 0.05	0.57 ± 0.01
95R+5M	1 ± 0	0.13 ± 0.01	0.05 ± 0.03	0.78 ± 0.01	0.93 ± 0.01	0.35 ± 0.04	0.54 ± 0.01
Kromaia oracle	1	0.41	0.10	0.91	0.97	0.11	0.58

Table 2

Holm's post hoc *p*Values for each quality metric (columns) and each pair of seed strategies (rows) whose value is above the threshold (0.05). Missing pairs of seed strategies obtain values below the threshold and are omitted for legibility.

	Duration	Uncertainty	Killer moves	Permanence	Lead change	Overall
100R vs. 1F	$\ll 2.2 \times 10^{-16}$	$\ll 2.2 \times 10^{-16}$	$\ll 2.2 \times 10^{-16}$	0.116	$\ll 2.2 \times 10^{-16}$	$\ll 2.2 \times 10^{-16}$
100R vs. 95R+5M	$\ll 2.2 \times 10^{-16}$	0.36651	1.3×10^{-6}	2.3×10^{-12}	0.0033	1
1M vs. 5F	$\ll 2.2 \times 10^{-16}$	0.36651	0.092	$\ll 2.2 \times 10^{-16}$	2.1×10^{-9}	$\ll 2.2 \times 10^{-16}$
1M vs. 5M	$\ll 2.2 \times 10^{-16}$	0.36651	0.726	0.998	0.8621	5.3×10^{-6}
1M vs. 95R+5F	0.367	8.6×10^{-7}	$\ll 2.2 \times 10^{-16}$	9.9×10^{-9}	$\ll 2.2 \times 10^{-16}$	1
95R+5F vs. 95R+5M	1.8×10^{-15}	8.8×10^{-16}	0.187	$\ll 2.2 \times 10^{-16}$	0.8621	1.8×10^{-11}

yielding better results than another one in terms of the quality measurements analysed.

The $\hat{A}_{12}$ between a pair of seed strategies is expressed as a value between 0 and 1 and indicates the probability of the first seed strategy yielding better results than the second. Table 3 shows the $\hat{A}_{12}$ results for each pair of seed strategies and measurement (i.e., if the results showed significant differences in the Holm's test). Extreme values indicate where the higher differences reside and are highlighted. The values above 90% are shown in dark grey, and the values below 10% are highlighted in light grey.

For instance, the fourth row (Table 3) is labelled as 100R vs. 5M, so each of the cells shows the percentage of times that applying the 100R seed strategy produces better results than applying the 5M seed strategy for a specific quality measurement. For instance, the value of Duration (first column) is 1.38%, so 100R yields better Duration values than 5M only 1.38% of the times. It is important to note that the values can be read both ways, so the 5M strategy produce better results than the 100R strategy for the Duration quality measurement 98.62% of the times.

To address Goal 2, which deals with determining the impact of the seed strategy on the quality of the boss models, we need **to answer Q3**, which asks about the quality of the models for each of the strategies studied: On average, the 5F seed strategy provides the best results (better than any other strategy 99% of the times), followed by the 1F strategy (around 80% of the times), followed by the 5M strategy (around 60% of the times), followed by 1M and 95R+5F (around 40% of the times), followed by the 100R (around 20% of the times) and the 95R+5M strategy (only around 12% of the times).

To address Goal 3, which deals with determining if the time needed by the developers to generate boss models is reduced by our approach, we need **to answer Q4 and Q5**, which ask about the time that the developers and our approach need in order to produce boss models, respectively: It was necessary to analyse the Version Control System used by the developers to determine the time that was originally spent to build the five final boss models. The sum of the time spent in the three development stages involved in this study (Spatial Organization, Behaviour Specification, and Equipment Balance) that led to the original final bosses that were commercially released was 10 months. Our approach needed approximately one hour for the

execution of each of the seed strategies. Therefore, to have a fair comparison, we would need to execute the EA five different times to generate five different final bosses, as in the original game, resulting in five hours. In other words, the approach is able to yield comparable results in less than a thousandth part of the original time required.

6. Discussion

Before conducting the experiment studied in this work, we thought that human-competitive results might be achievable by taking one or several final boss models as the starting point. This coincides with the idea of Genetic Improvement (Petke et al., 2018), for which the results are obtained using seeds that are similar enough to solutions. Paradoxically, the main disadvantage associated to using final boss models as seeds is that it is necessary to obtain those models in advance, and it is time-consuming for humans to generate such complete models. A random sample taken from the results suggested that using final boss models as seeds could lead to final bosses that are very similar to those seeds, which is another disadvantage if the final bosses only provide small variations instead of new, varied video game content that is found to be engaging and not repetitive by users.

We also used miniboss models as seeds. These models include over 500 model elements, whereas a final boss model could involve around 1300 model elements. Using various different miniboss models as seeds shows that the final boss models obtained include characteristics found in the seeds. These bosses, which are significantly less complex to design, could be useful for controlling the characteristics in the final boss models generated. However, we must study this possibility carefully in future works.

We also tested our approach with an initial population that consisted of models that were generated randomly. The combination of our fitness and repair operations makes it possible for these models to evolve in order to achieve models that are comparable to those provided by the developers. This coincides with the idea of Genetic Programming (Goldberg, 1983), which generates a complete program using genetic encoding. In our case, this is Genetic Software Modelling since, in our work, we deal with software models. The results obtained in our work do not claim that it is possible to obtain a complete model of a whole

Table 3

The $\hat{A}_{12}$ statistic for each quality metric (columns) and pair of seed strategies (rows) with significant differences according to the Holms post hoc. Values above 90% are highlighted in dark grey and values below 10% are highlighted in light grey.

	Duration	Uncertainty	Killer moves	Permanence	Lead change	Overall
100R vs. 1M	98.6%	1.68%	0%	95.09%	93.88%	23.04%
100R vs. 5F	0%	3.16%	0%	77.11%	65.7%	3.04%
100R vs. 5M	1.38%	2.74%	0%	95.19%	96.26%	22.68%
100R vs. 95R+5M	100%	45.72%	25.82%	82.74%	23.24%	29.31%
1F vs. 1M	100%	88.92%	100%	99.97%	0%	99.72%
1F vs. 5F	0%	75.90%	100%	88.42%	0%	2.67%
1F vs. 5M	72.06%	91.73%	100%	99.92%	0%	94.35%
1F vs. 95R+5F	100%	96.79%	100%	98.75%	0%	98.39%
1F vs. 95R+5M	100%	100%	100%	92.27%	0%	99.95%
1M vs. 95R+5M	99.69%	100%	100%	15.29%	0%	98.26%
5F vs. 5M	100%	59.51%	65.72%	96.26%	84.82%	99.86%
5F vs. 95R+5F	100%	76.89%	100%	96.05%	0.74%	99.98%
5F vs. 95R+5M	100%	100%	100%	68.26%	0.06%	100%
5M vs. 95R+5F	99.76%	74.84%	100%	82.76%	0.19%	82.68%
5M vs. 95R+5M	100%	100%	100%	14.71%	0.02%	99.27%

video game with our EMOGen approach. However, our results do show that it is possible to perform Genetic Software Modelling to achieve results that are comparable to the bosses in the commercial releases of Kromaia. This is feasible because we apply our approach to models, which have less noise than source code because software models abstract from implementation details.

An issue to be addressed in the future is the use of our approach in other industrial contexts. Commercial engines, like Unity (U. Technologies, 2005) or Unreal, which uses its own DSL named BluePrints (Games, 1998), are widely adopted by development teams, and their architecture is similar to that shown in Fig. 4. These DSLs are similar to SDML in terms of level of detail, and allow for the description of every element present in a game. Since the ideas proposed in this work are general, their application to different commercial DSLs is part of our future work.

7. Threats to validity

Following the guidelines suggested by De Oliveira Barros and Dias-Neto (2011), we have identified the following threats to validity:

Not accounting for random variation: We addressed this threat by performing 30 runs for each of the executions of our approach.

Lack of a meaningful comparison baseline: We addressed this threat by comparing our approach with a random search and also by comparing the results with the final bosses from the developers of the commercial release of the video game case study.

Lack of clarity on data collection: We addressed this threat by using the data provided by the SDML models of the contenders to perform the simulation and two main indicators that were obtained from the developers and used to give value to configurations: victory percentage and health level.

Lack of real problem instances: The case study used in the evaluation is an industrial video game, and the problem artefacts were directly obtained from the video game industry.

Lack of assessing the validity of cost measures: We performed a fair comparison between the final bosses from Kromaia and the bosses generated by our approach, studying the time spent by the developers and our algorithms to obtain the results.

Lack of assessing effective measurements: We addressed this threat by using quality measures that are presented in the literature of video game research (Browne and Maire, 2010).

8. Conclusion

Our EMOGen approach produces content for video games, whole models of final bosses that could be used in Kromaia, the

video game case study. The production of these models is relevant for the creation of the video game, and processes like updates or expansions, which are demanding in terms of quality and release schedule.

The quality of the bosses obtained by our approach is comparable to that achieved by the professional video game developers that produced the final bosses that were included in the commercial release of the case study.

The results show that the seeds used, the final boss models which the evolutionary algorithm of our approach is fed with as starting points, have an impact in the quality of the bosses produced: the use of the final bosses or minibosses included in the commercial video game case study helps our approach obtain boss models of higher quality in comparison to those produced when the seeds are random models.

EMOGen, which uses DSL models, propose ideas which do not make our approach depend on the video game studied in this work. Therefore, the applicability of our approach could be studied in the context of other commercial frameworks.

Our approach only takes five hours of unattended time in comparison with ten months of work by the video game developers. Our work also offers a relevant result for genetic software modelling since human-competitive software models can even be achieved from randomly generated models, i.e., without a starting modelling effort from developers. We have made two model examples and an implementation of EMOGen freely available in order to facilitate the adoption of our approach.

CRedit authorship contribution statement

Daniel Blasco: Conceptualization, Software, Validation, Methodology, Formal analysis, Investigation, Resources, Data curation, Writing - original draft, Writing - review & editing, Supervision. **Jaime Font:** Methodology, Formal analysis, Investigation, Resources, Data curation, Visualization. **Mar Zamorano:** Methodology, Formal analysis, Investigation, Resources, Data curation, Visualization. **Carlos Cetina:** Conceptualization, Investigation, Resources, Writing - review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work has been partially supported by the Ministry of Economy and Competitiveness (MINECO) through the Spanish National R+D+i Plan and ERDF funds under the Project ALPS (RTI2018-096411-B-I00).

References

- Althöfer, I., 2003. Computer-Aided Game Inventing. Technical Report, Friedrich Schiller Universität Jena.
- Ampatzoglou, A., Stamelos, I., 2010. Software engineering research for computer games: A systematic review. *Inf. Softw. Technol.* 52 (9), 888–901.
- Arcega, L., Font, J., Cetina, C., 2018. Evolutionary algorithm for bug localization in the reconfigurations of models at runtime. In: *Proceedings of the 21th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS 2018, Copenhagen, Denmark, October 14–19, 2018*. pp. 90–100. <http://dx.doi.org/10.1145/3239372.3239392>.
- Arcega, L., Font, J., Haugen, Ø., Cetina, C., 2017. On the influence of models at runtime traces in dynamic feature location. In: Anjorin, A., Espinoza, H. (Eds.), *Modelling Foundations and Applications*. Springer International Publishing, Cham, pp. 90–105.
- Arcega, L., Font, J., Haugen, Ø., Cetina, C., 2019. An approach for bug localization in models using two levels: model and metamodel. *Softw. Syst. Model.*
- Arcuri, A., Briand, L., 2014. A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. *Softw. Test. Verif. Reliab.* 24 (3), 219–250.
- Arcuri, A., Fraser, G., 2013. Parameter tuning or default values? An empirical investigation in search-based software engineering. *Empir. Softw. Eng.* 18 (3), 594–623.
- Ashlock, D., 2010. Automatic generation of game elements via evolution. In: *Proceedings of the 2010 IEEE Conference on Computational Intelligence and Games, IEEE*. pp. 289–296.
- Ashlock, D.A., Gent, S.P., Bryden, K.M., 2005. Evolution of l-systems for compact virtual landscape generation. In: *2005 IEEE Congress on Evolutionary Computation, Vol. 3, IEEE*. pp. 2760–2767.
- Ashlock, D., Lee, C., McGuinness, C., 2011. Search-based procedural generation of maze-like levels. *IEEE Trans. Comput. Intell. AI Games* 3 (3), 260–273.
- Basili, V.R., 1996. The Role of Experimentation in Software Engineering: Past, Current, and Future. Berlin, Germany. pp. 442–449.
- Basili, V.R., Caldiera, G., Rombach, H.D., 1994. The goal question metric approach. In: *Encyclopedia of Software Engineering*. John Wiley & Sons.
- Bergstra, J., Bengio, Y., 2012. Random search for hyper-parameter optimization. *J. Mach. Learn. Res.* 13 (1), 281–305.
- Birkhoff, G.D., 1933. *Aesthetic Measure*. Harvard University Press, Cambridge, Massachusetts.
- Browne, C., 2005. *Connection Games: Variations on a Theme*. AK Peters, Natick, Massachusetts.
- Browne, C.B., 2008. *Automatic Generation and Evaluation of Recombination Games* (Ph.D. thesis). Queensland University of Technology.
- Browne, C., Maire, F., 2010. Evolutionary game design. *IEEE Trans. Comput. Intell. AI Games* 2 (1), 1–16.
- De Oliveira Barros, M., Dias-Neto, A.C., 2011. 0006/2011-threats to validity in search-based software engineering empirical studies. *RelaTe-DIA* 5 (1).
- Ellis, H., 1914. *Impressions and Comments*. Houghton Mifflin, Boston, Massachusetts.
- Font, J., Arcega, L., Haugen, Ø., Cetina, C., 2016. Feature location in models through a genetic algorithm driven by information retrieval techniques. In: *Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems, MODELS '16, ACM, New York, NY, USA*, pp. 272–282. <http://dx.doi.org/10.1145/2976767.2976789>, URL: <http://doi.acm.org/10.1145/2976767.2976789>.
- Frade, M., de Vega, F.F., Cotta, C., 2010. Evolution of artificial terrains for video games based on accessibility. In: *European Conference on the Applications of Evolutionary Computation*. Springer, pp. 90–99.
- Games, E., 1998. Unreal engine, version 2018.3.9. URL: <http://www.unrealengine.com/>.
- García, S., Fernández, A., Luengo, J., Herrera, F., 2010a. Advanced nonparametric tests for multiple comparisons in the design of experiments in computational intelligence and data mining: Experimental analysis of power. *Inform. Sci.* 180 (10), 2044–2064, Special Issue on Intelligent Distributed Information Systems.
- García, S., Fernández, A., Luengo, J., Herrera, F., 2010b. Advanced nonparametric tests for multiple comparisons in the design of experiments in computational intelligence and data mining: Experimental analysis of power. *Inform. Sci.* 180 (10), 2044–2064.
- Gardner, M., 2004. *Theory of everything*. New Criterion 23 (2).
- Goldberg, D.E., 1983. *Computer-Aided Gas Pipeline Operation using Genetic Algorithms* (Ph.D. dissertation). University of Michigan.
- Goldsby, H.J., Cheng, B.H.C., 2008. Automatically generating behavioral models of adaptive systems to address uncertainty. In: *Czarnecki, K., Ober, I., Bruehl, J.-M., Uhl, A., Völter, M. (Eds.), Model Driven Engineering Languages and Systems*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 568–583.
- Grissom, R.J., Kim, J.J., 2005. *Effect Sizes for Research: A Broad Practical Approach*. Earlbaum, Mahwah, NJ.
- Harman, M., Jones, B., 2001. Search-based software engineering. *Inf. Softw. Technol.* 43, 833–839.
- Hastings, E.J., Guha, R.K., Stanley, K.O., 2009. Evolving content in the galactic arms race video game. In: *2009 IEEE Symposium on Computational Intelligence and Games, IEEE*. pp. 241–248.
- Hastings, E.J., Stanley, K.O., 2010. Interactive genetic engineering of evolved video game content. In: *Proceedings of the 2010 Workshop on Procedural Content Generation in Games, ACM*, p. 8.
- Hom, V., Marks, J., 2007. Automatic design of balanced board games. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*. pp. 25–30.
- IGDA, International Game Developers Association, 2018. URL: <http://tiny.cc/ucev5y>.
- Iida, H., Takahara, K., Nagashima, J., Kajihara, Y., Hashimoto, T., 2004. An application of game-refinement theory to mah jong. In: *Entertainment Computing - ICEC 2004, Third International Conference, Eindhoven, the Netherlands, September 1–3, 2004, Proceedings*. pp. 333–338. http://dx.doi.org/10.1007/978-3-540-28643-1_41.
- Jennings-Teats, M., Smith, G., Wardrip-Fruin, N., 2010. Polymorph: A model for dynamic level generation. In: *Sixth Artificial Intelligence and Interactive Digital Entertainment Conference*.
- Johnson, D.S., 2002. A theoretician's guide to the experimental analysis of algorithms. In: *Data Structures, Near Neighbor Searches, and Methodology: Fifth and Sixth DIMACS Implementation Challenges, Vol. 59*. pp. 215–250.
- Kent, S., 2002. Model driven engineering. In: *Butler, M., Petre, L., Sere, K. (Eds.), Integrated Formal Methods*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 286–298.
- Koza, J.R., 1992. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA.
- Kramer, W., 2000. What makes a game good? Games J.
- Lelis, L.H., Reis, W.M., Gal, Y., 2017. Procedural generation of game maps with human-in-the-loop algorithms. *IEEE Trans. Games* 10 (3), 271–280.
- Loiacono, D., Cardamone, L., Lanzi, P.L., 2011. Automatic track generation for high-end racing games using evolutionary computation. *IEEE Trans. Comput. Intell. AI Games* 3 (3), 245–259.
- Martin, A., Lim, A., Colton, S., Browne, C., 2010. Evolving 3d buildings for the prototype video game subversion. In: *European Conference on the Applications of Evolutionary Computation*. Springer, pp. 111–120.
- McShaffry, M., 2003. *Game Coding Complete*. Paraglyph Publishing.
- Meta Object Facility, 2013. *Meta Object Facility (MOF) Version 2.4.1. Object Management Group (OMG) Specification*.
- Nogueira-Collazo, M., Porras, C.C., Fernández-Leiva, A.J., 2015. Competitive algorithms for coevolving both game content and AI. A case study: Planet wars. *IEEE Trans. Comput. Intell. AI Games* 8 (4), 325–337.
- Núñez-Valdéz, E.R., García-Díaz, V., Lovelle, J.M.C., Achaerandio, Y.S., Crespo, R.G., 2017. A model-driven approach to generate and deploy videogames on multiple platforms. *J. Ambient Intell. Humaniz. Comput.* 8 (3), 435–447.
- Núñez-Valdéz, E.R., Martínez, O.S., García-Bustelo, B.C.P., Lovelle, J.M.C., Infante-Hernandez, G., 2013. Gade4all: Developing multi-platform videogames based on domain specific languages and model driven engineering. *IJMAI* 2 (2), 33–42.
- Oranchak, D., 2010. Evolutionary algorithm for generation of entertaining shinro logic puzzles. In: *European Conference on the Applications of Evolutionary Computation*. Springer, pp. 181–190.
- Pedersen, C., Togelius, J., Yannakakis, G.N., 2009. Modeling player experience in super mario bros. In: *2009 IEEE Symposium on Computational Intelligence and Games, IEEE*. pp. 132–139.
- Petke, J., Haraldsson, S.O., Harman, M., Langdon, W.B., White, D.R., Woodward, J.R., 2018. Genetic improvement of software: A comprehensive survey. *IEEE Trans. Evol. Comput.* 22 (3), 415–432.
- Raffe, W.L., Zambetta, F., Li, X., Stanley, K.O., 2014. Integrated approach to personalized procedural map generation using evolutionary algorithms. *IEEE Trans. Comput. Intell. AI Games* 7 (2), 139–155.
- Reyno, E.M., Carsí Cubel, J.A., 2009. Automatic prototyping in model-driven game development. *Comput. Entertain.* 7 (2), 29:1–29:9.
- Risi, S., Lehman, J., D'Ambrosio, D.B., Hall, R., Stanley, K.O., 2015. Petalz: Search-based procedural content generation for the casual gamer. *IEEE Trans. Comput. Intell. AI Games* 8 (3), 244–255.
- Roberts, J., Chen, K., 2014. Learning-based procedural content generation. *IEEE Trans. Comput. Intell. AI Games* 7 (1), 88–101.
- Salge, C., Mahlmann, T., 2010. Relevant information as a formalised approach to evaluate game mechanics. In: *Proceedings of the 2010 IEEE Conference on Computational Intelligence and Games, IEEE*. pp. 281–288.
- Selic, B., 2003. The pragmatics of model-driven development. *IEEE Softw.* 20 (5), 19–25.

- Shull, F., Singer, J., Sjberg, D.I.K., 2010. Guide to Advanced Empirical Software Engineering, first ed. Springer Publishing Company, Incorporated.
- Smith, A.M., Mateas, M., 2010. Variations forever: Flexibly generating rulesets from a sculptable design space of mini-games. In: Proceedings of the 2010 IEEE Conference on Computational Intelligence and Games. IEEE, pp. 273–280.
- Smith, A.M., Mateas, M., 2011. Answer set programming for procedural content generation: A design space approach. *IEEE Trans. Comput. Intell. AI Games* 3 (3), 187–200.
- Solís-Martínez, J., Espada, J.P., García-Menéndez, N., García-Bustelo, B.C.P., Lovelle, J.M.C., 2015. VGPM: using business process modeling for videogame modeling and code generation in multiple platforms. *Comput. Stand. Interfaces* 42, 42–52.
- Sorenson, N., Pasquier, P., 2010. Towards a generic framework for automated video game level creation. In: European Conference on the Applications of Evolutionary Computation. Springer, pp. 131–140.
- Sorenson, N., Pasquier, P., DiPaola, S., 2011. A generic approach to challenge modeling for the procedural creation of video game levels. *IEEE Trans. Comput. Intell. AI Games* 3 (3), 229–244.
- Stiny, G., Gips, J., 1978. Algorithmic Aesthetics: Computer Models for Criticism and Design in the Arts. University of California Press, Berkeley, California.
- Summerville, A., Snodgrass, S., Guzdial, M., Holmgård, C., Hoover, A.K., Isaksen, A., Nealen, A., Togelius, J., 2018. Procedural content generation via machine learning (PCGML). *IEEE Trans. Games* 10 (3), 257–270.
- Thompson, J.M., 2000. Defining the abstract. *Games J.*
- Togelius, J., De Nardi, R., Lucas, S.M., 2006. Making racing fun through player modeling and track evolution.
- Togelius, J., Preuss, M., Yannakakis, G.N., 2010. Towards multiobjective procedural map generation. In: Proceedings of the 2010 Workshop on Procedural Content Generation in Games. ACM, p. 3.
- Togelius, J., Schmidhuber, J., 2008. An experiment in automatic game design. In: 2008 IEEE Symposium on Computational Intelligence and Games. IEEE, pp. 111–118.
- Togelius, J., Yannakakis, G.N., Stanley, K.O., Browne, C., 2011. Search-based procedural content generation: A taxonomy and survey. *IEEE Trans. Comput. Intell. AI Games* 3 (3), 172–186.
- Travassos, G.H., de Oliveira Barros, M., 2003. Contributions of in virtuo and in silico experiments for the future of empirical studies in software engineering. In: Proceedings of the ESEIW 2003 Workshop on Empirical Studies in Software Engineering (WSESE '03). IEEE Computer Society, Roman Castles, Italy.
- Tutenel, T., Smelik, R.M., Lopes, R., De Kraker, K.J., Bidarra, R., 2011. Generating consistent buildings: a semantic approach for integrating procedural techniques. *IEEE Trans. Comput. Intell. AI Games* 3 (3), 274–288.
- U. Technologies, 2005. Unity, version 2018.3.9. URL: <https://unity.com/>.
- Usman, M., Iqbal, M.Z., Khan, M.U., 2017. A product-line model-driven engineering approach for generating feature-based mobile applications. *J. Syst. Softw.* 123, 1–32.
- Van Der Linden, R., Lopes, R., Bidarra, R., 2013. Procedural generation of dungeons. *IEEE Trans. Comput. Intell. AI Games* 6 (1), 78–89.
- Vargha, A., Delaney, H.D., 2000. A critique and improvement of the CL common language effect size statistics of mcgraw and wong. *J. Educ. Behav. Stat.* 25 (2), 101–132.
- Williams, J.R., Poulding, S.M., Rose, L.M., Paige, R.F., Polack, F.A.C., 2011. Identifying desirable game character behaviours through the application of evolutionary algorithms to model-driven engineering metamodels. In: Search Based Software Engineering - Third International Symposium, SSBSE 2011, Szeged, Hungary, September 10–12, 2011. Proceedings. pp. 112–126. http://dx.doi.org/10.1007/978-3-642-23716-4_13.

Daniel Blasco received the B.S. degree in computer science from Universidad de Zaragoza, Zaragoza, Spain. He is currently pursuing the Ph.D. degree in computer science with the Universitat Politècnica de València, Valencia, Spain. He is a Researcher with the SVIT Research Group, San Jorge University. His current research interests include Game Software engineering, Traceability Link Recovery, and Procedural Content Generation.

Jaime Font received the Ph.D. degree in computer science from University of Oslo, Oslo, Norway. He is an Assistant Professor with the SVIT Research Group, Universidad San Jorge, Zaragoza, Spain. His current research interests include reverse engineering, evolutionary computation, and variability modelling.

Mar Zamorano received the M.Sc. degree in advanced software technologies from San Jorge University, Zaragoza, Spain. She is a Researcher with the SVIT Research Group, San Jorge University. Her current research interests include Game Software engineering, and Procedural Content Generation.

Carlos Cetina is an associate professor with San Jorge University and the Head of the SVIT Research Group. He received a Ph.D. in computer science from the Technical University of Valencia. His research focuses on software product lines and model-driven development. His research results have reshaped software development in world-leading industries from heterogeneous domains ranging from induction hob firmware to train control and management systems. More information about his background can be found at his website: www.carloscetina.com.